



Clase 11

Observabilidad con OpenTelemetry

Andres Caicedo · AI for Developers | 30X · 1h · 60 min

¿Dónde estamos en el journey?

Funciona hoy. ¿Cómo sabes que sigue funcionando mañana?

- ▶ **Clase 8** → QA funcional aprobado
- ▶ **Clase 9** → infraestructura desplegada en la nube con IaC
- ▶ **Clase 10** → seguridad validada: SAST · DAST · agentic security
- ▶ **El pipeline de construcción está cerrado** — hoy empieza el ciclo de operación
- ▶ **Hoy:** stack de observabilidad con Docker Compose + instrumentación del chat de IA

 Que el producto funcione ahora no garantiza que funcione mañana. Observabilidad es ver lo que pasa antes de que alguien te lo reporte.



DEMO

Abrir EntreVista AI en localhost:3000 → navegar campañas → una conversación con el chat → todo funciona → ¿cómo detectas un problema antes de que un usuario lo reporte?

02 — SECCIÓN

OpenTelemetry y sus Señales

Un estándar abierto para ver lo que pasa dentro de tu aplicación

¿Qué es OpenTelemetry?

Estándar abierto de observabilidad — una instrumentación, múltiples backends

- ▶ **Origen:** fusión de OpenTracing + OpenCensus → proyecto de la Cloud Native Computing Foundation(CNCF)
- ▶ **Vendor-neutral:** la misma instrumentación sirve para Jaeger, Grafana, Datadog o New Relic
- ▶ **Componentes:** SDK instrumenta · API estandariza · Collector recibe y distribuye · Exporters envían
- ▶ **Adopción:** segundo proyecto más activo de la CNCF después de Kubernetes

💡 OpenTelemetry no es un producto de monitoring — es el estándar que define cómo tu app emite telemetría. El backend lo eliges tú.



DEMO

Abrir opentelemetry.io/docs → arquitectura del Collector → seguir el camino SDK → Collector → múltiples backends

¿Qué es OpenTelemetry?

Estándar abierto de observabilidad — una instrumentación, múltiples backends

- ▶ **Origen:** fusión de OpenTracing + OpenCensus → proyecto de la Cloud Native Computing Foundation(CNCF)
- ▶ **Vendor-neutral:** la misma instrumentación sirve para Jaeger, Grafana, Datadog o New Relic
- ▶ **Componentes:** SDK instrumenta · API estandariza · Collector recibe y distribuye · Exporters envían
- ▶ **Adopción:** segundo proyecto más activo de la CNCF después de Kubernetes

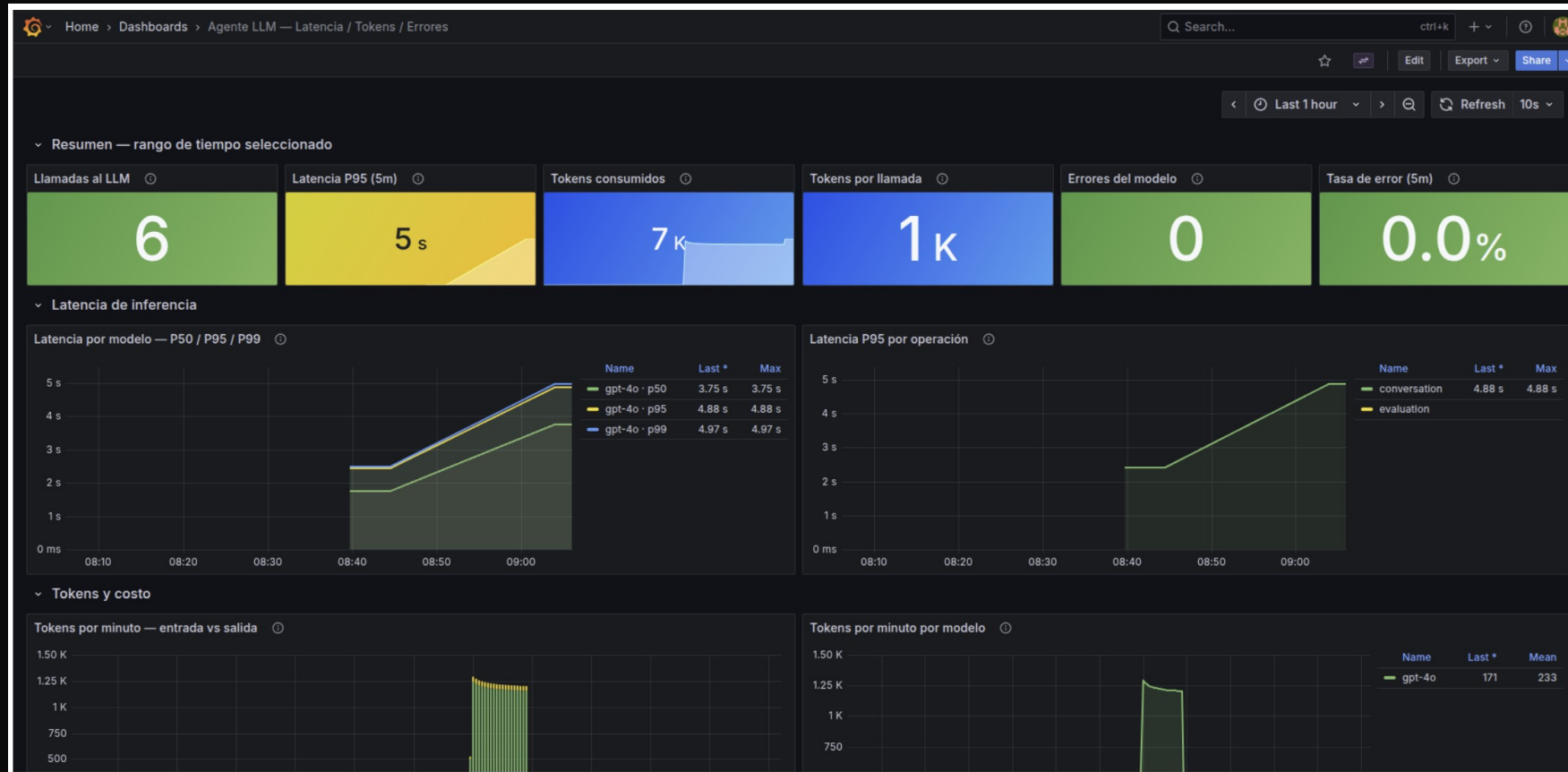
 OpenTelemetry no es un producto de monitoring — es el estándar que define cómo tu app emite telemetría. El backend lo eliges tú.



DEMO

Abrir opentelemetry.io/docs → arquitectura del Collector → seguir el camino SDK → Collector → múltiples backends

Señales : Negocio

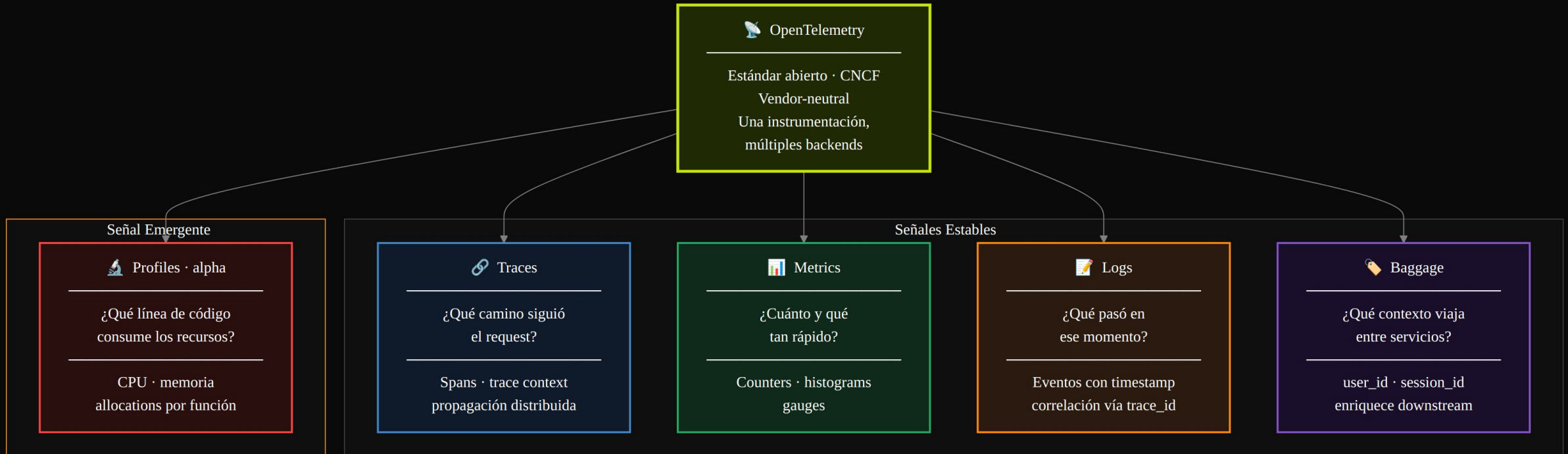


Señales : Técnicas



Las 5 señales de OpenTelemetry

4 estables + 1 emergente — cada una responde una pregunta diferente



El Collector como hub central

Receivers → Processors → Exporters — buffering, retry, multi-destino

- ▶ **Receiver OTLP:** recibe toda la telemetría en un protocolo estándar — gRPC o HTTP
- ▶ **Processors:** batch, retry, filtrado y transformación antes de exportar
- ▶ **Exporters:** envía a varios backends a la vez — Tempo, Prometheus, Loki
- ▶ **Por qué no exportar directo:** absorbe picos, reintenta y desacopla la app del backend

 Cambias Jaeger por Tempo y solo tocas la config del Collector. Tu instrumentación sobrevive al cambio de backend.

03 — SECCIÓN

¿Qué es Diferente con IA Generativa?

Las métricas de HTTP status y latencia ya no son suficientes

Dimensiones nuevas que no existen en apps tradicionales

Latencia en segundos, costo por token, calidad no determinística

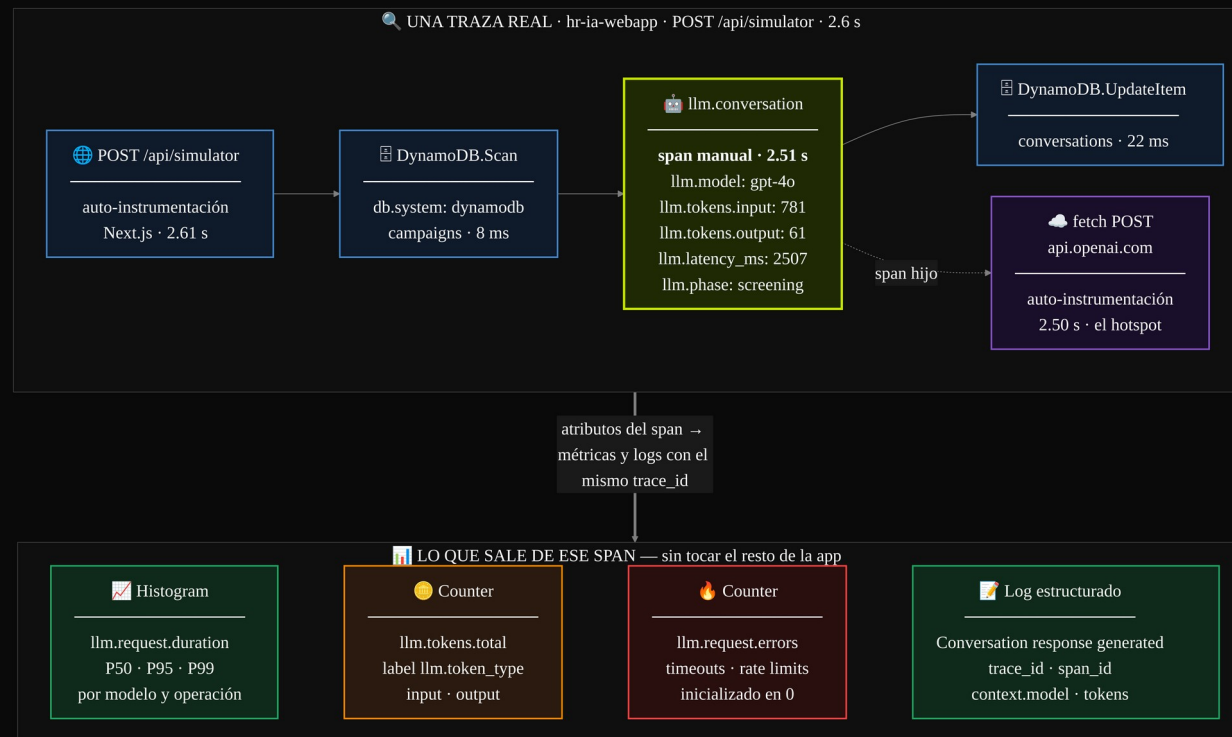
- ▶ **Latencia de inferencia:** segundos, no milisegundos — un span de LLM puede durar 5 a 30 s
- ▶ **Costo por token:** cada conversación tiene precio directo — entrada + salida
- ▶ **Calidad no determinística:** la misma pregunta produce respuestas de calidad variable
- ▶ **Alucinaciones:** el modelo inventa información — no hay HTTP 500 que lo detecte
- ▶ **Adherencia al system prompt:** el modelo puede desviarse de las instrucciones
 - * Librerías para explorar: Ragas, DeepEval,

💡 Un HTTP 200 con una alucinación es un fallo invisible. La observabilidad de IA necesita métricas que el status code no captura.

<https://deepeval.com/blog/top-5-llm-evaluation-frameworks>

Instrumentación de conversaciones con LLMs

Cada llamada al modelo es un span con atributos específicos de IA

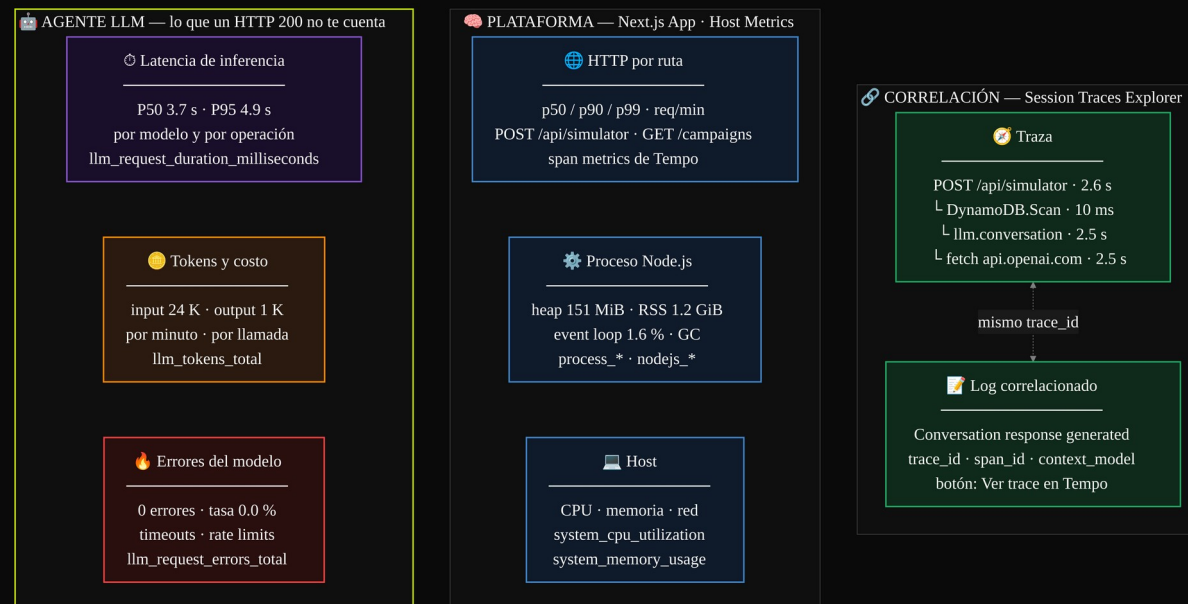


DEMO

Abrir `chat-client.ts` en `requirement-hr-ia` → span `llm.conversation` con `llm.model` · `llm.tokens.input` · `llm.tokens.output` · `llm.latency_ms` → las 3 métricas: `llm.request.duration` · `llm.tokens.total` · `llm.request.errors`

Dashboard de observabilidad

Traces, métricas y logs correlacionados en un solo panel



💡 Los dashboards son código: cuatro JSON en el repo de la app que Grafana carga al arrancar. No son el objetivo — sin spans y métricas bien definidos, el dashboard está vacío.

Uniendo lo técnico con el negocio

Un error 500 no dice cuánto dinero se perdió

- ▶ **El problema:** ves latencia o errores, pero no sabes si cayó una venta o un carrito de prueba
- ▶ **Enriquecer spans:** atributos de negocio en operaciones existentes — ``user.id``, ``plan.type``, ``cart.value``
- ▶ **Eventos de negocio:** Span Events para momentos puntuales — "Checkout completado", "Registro fallido"
- ▶ **Métricas de negocio:** contadores como ``ventas.total.count`` o ``revenue.per_hour``

💡 OpenTelemetry instrumenta la lógica de negocio dentro del mismo flujo técnico: dashboards donde filtras problemas técnicos por criticidad de negocio.

TRANSICIÓN

Práctica demostrativa

Del concepto a la ejecución — dos repositorios listos para replicar

- ▶ ``otel-stack``: Collector + Tempo + Prometheus + Loki + Grafana en un ``docker compose up -d``
- ▶ ``requirement-hr-ia``: EntreVista AI instrumentado — traces automáticos, spans del LLM, logs correlacionados
- ▶ Dashboards como código: cuatro JSON provisionados en Grafana, incluido el del agente LLM

Para la próxima sesión

Observabilidad de extremo a extremo en tu producto

- ▶ ☐ **Stack levantado** — clona `otel-stack` y ejecuta `docker compose up -d` en tu máquina
- ▶ ☐ **App instrumentada** — auto-instrumentación HTTP + spans manuales para el LLM, con `requirement-hr-ia` de referencia
- ▶ ☐ **3 métricas del agente** — latencia de inferencia, tokens consumidos y tasa de error del modelo
- ▶ ☐ **Verificación en Grafana** — traces en Tempo y métricas en Prometheus de tu app
- ▶ ☐ **Dashboard como código + skill** — un JSON cargado con `DASHBOARDS\_DIR` y el skill ajustado a tu producto



Es lo último que entra al producto antes del Demo Day: la evidencia de que sabes qué pasa en producción, no solo que funciona en tu máquina.